

به نام خدا



امنیت داده و شبکه

نیم سال دوم ۱۴۰۴-۱۴۰۵

دانشکده‌ی مهندسی کامپیوتر

دانشگاه صنعتی شریف

موضوع بهره‌برداری از آسیب‌پذیری برنامه‌ها

موعده تحویل ساعت ۲۳:۵۹، ۲۲ اردیبهشت ۱۴۰۵

طراحی تمرین توسط محمد حدادیان، رضا سعیدی، محمدفرحان بهرامی و محمدعلی کاکاوند

مقدمه

هدف از این تمرین تجربه بیشتر در شناسایی و بهره‌برداری از آسیب‌پذیری‌های برنامه‌ها است. این تمرین از چهار بخش تشکیل شده است. برای سه بخش اول شما باید ابتدا آسیب‌پذیری برنامه‌ی داده‌شده را پیدا کرده و سپس با نوشتن یک اسکریپت (به هر زبان دلخواه) از آن آسیب‌پذیری سوءاستفاده کرده، به shell دست یابید. در بخش‌های دوم و سوم تمرین بعد از دستیابی به شل، به پرچم موجود روی ماشین هدف دسترسی پیدا کنید (هدف اصلی شما دسترسی به پرچم است، گاهی برای دسترسی نیاز است تا دسترسی کامل شل بگیرید و گاهی خواندن محتوای پرچم بدون دستیابی کامل به شل امکان‌پذیر است). پرچم هر بخش به صورت یک رشته به فرمت `CE441{xxxx}` می‌باشد.

۱ بخش اول

۱.۱ راه‌اندازی محیط

برای بخش اول این تمرین، یک ماشین مجازی^۱ در اختیار شما قرار گرفته است. در بهره‌برداری از آسیب‌پذیری‌ها، همه چیز از نسخه‌ی کامپایلر تا مکانیزم‌های امنیتی سیستم‌عامل دخیل خواهند بود. با داشتن این ماشین مجازی، در اجرای اکسپلویت‌های خود یکپارچه خواهید بود. برای راه‌اندازی این ماشین مجازی نیاز به مجازی‌ساز qemu خواهید داشت. روش نصب آن بسته به سیستم‌عامل شما متفاوت است:

- ویندوز: فایل نصب‌کننده را از این لینک^۲ دریافت کنید.

- لینوکس: با دستور `sudo apt install qemu-system-x86` نصب کنید.

- macOS: با دستور `brew install qemu` نصب کنید.

پس از دانلود ماشین مجازی و نصب qemu، به ترتیب مراحل زیر را اجرا کنید تا ماشین مجازی شما آماده استفاده شود.

۱. با استفاده از دستور `tar xvf proj1-vm.tar.gz` ماشین مجازی را استخراج کنید. پس از استخراج، اسنپ‌شات ماشین مجازی به نام `disk.img` را به همراه دو اسکریپت کمکی خواهید دید.

۲. در ادامه برای اجرای ماشین مجازی لازم است دستور زیر را در ترمینال خود اجرا کنید. این دستور ماشین مجازی مورد نظر را با ۲ گیگابایت رم اجرا می‌کند و پورت ۲۲ آنرا به پورت ۵۵۵۵ ماشین شما منتقل می‌کند.

```
1 qemu-system-x86_64 disk.img -m 2G -nic user,hostfwd=tcp::5555-:22 -nographic
```

۳. در نهایت با اجرای دستور `ssh user@localhost -p 5555` به ترمینال ماشین مجازی با نام کاربری `user` و رمز عبور `cs155` متصل شوید و از آن استفاده کنید.

۲.۱ اهداف

در پوشه `/targets` کد منبع پنج هدف آسیب‌پذیر به همراه `Makefile` برای کامپایل و اجرای آن‌ها قرار دارد. در بخش اول این تمرین باید هر یک از این اهداف را مورد بهره‌برداری قرار دهید؛ هر هدف دارای نوعی آسیب‌پذیری است و هدف نهایی شما دستیابی به شل با دسترسی root با اجرای برنامه‌ها و ارسال ورودی مناسب است. برای کامپایل و نصب این اهداف دستورهای زیر را اجرا کنید.

^۱ <https://my.sharif.edu/s/wd3nwmWos3ibRFj>

^۲ <https://my.sharif.edu/s/BDtWLFKFSJxeoCR>

```

1 cd targets
2 make
3 sudo make install

```

با اجرای این دستورات، فایل‌های اجرایی اهداف در آدرس `/tmp` قرار می‌گیرند. دقت کنید که اکسپلویت شما باید این اهداف را دقیقاً در پوشه‌ی `/tmp` اجرا و بهره‌برداری کند.

۱.۲.۱ اهداف اول تا سوم

بهره‌برداری از این اهداف نیازمند دانش کافی درباره‌ی چگونگی فراخوانی و بازگشت توابع و رفتار پشته است. همچنین برای درک بهتر و ساده‌تر شدن فرایند اجرای کد می‌توانید از دیباگرهایی مانند `gdb` استفاده کنید. باینری‌های هدف با فلگ `-z execstack` کامپایل شده‌اند و پشته‌ای با قابلیت اجرای کد دارند، این ویژگی باعث می‌شود حملاتی مانند `buffer overflow` قابل تمرین و بررسی باشند. در تمامی این اهداف نوعی آسیب‌پذیری سرریز بافر وجود دارد که باید مورد تحلیل و بهره‌برداری قرار گیرد.

۲.۲.۱ هدف چهارم

این هدف بر اساس بهره‌برداری از `GOT`^۳ و `PLT`^۴ است. در این نوع بهره‌برداری، از این جداول برای تغییر آدرس‌های حافظه و دستکاری فراخوانی توابع در زمان اجرا استفاده می‌شود. برای اطلاعات بیشتر در مورد این تکنیک و نحوه عملکرد آن، به فایل `got-plt-exploitation.txt` موجود در پوشه‌ی تمرین مراجعه کنید.

۳.۲.۱ هدف پنجم

در این هدف برخلاف اهداف قبلی، پشته برنامه قابلیت اجرای کد ندارد و شما برای دسترسی به شل باید روش دیگری به جز اجرای کد در پشته پیدا کنید. همچنین در ماشین مجازی‌ای که در اختیار شما قرار داده شده است، برنامه کمکی‌ای به نام `find_gadgets.py` وجود دارد که به شما در نوشتن این بهره‌برداری کمک می‌کند.

۳.۱ ساختار کد اکسپلویت

در ماشین مجازی، پوشه‌ای به نام `/xexploits` وجود دارد که شامل قالب اولیه کدهای بهره‌برداری است. شما باید با تکمیل این کدها و اجرای دستور `make` آن‌ها را کامپایل کرده و سپس اجرا کنید. توجه داشته باشید که این کدها بر روی هدف معادل خود در پوشه `/tmp` اجرا می‌شوند و ورودی‌هایی که در این کدها تولید کرده‌اید، به آن‌ها داده می‌شود. همچنین در فایل `shellcode.h` یک نمونه شل موجود است که می‌توانید از آن در اکسپلویت‌های خود استفاده کنید. در نهایت، انتظار می‌رود که با بهره‌برداری صحیح از آسیب‌پذیری‌ها و تکمیل درست کدهای موجود در این پوشه، به شل با دسترسی `root` مشابه نمونه‌ی زیر دست یابید.

```

1 user@cs155:/proj1/xexploits whoamiuser@cs155:/proj1/xexploits ./xploit1
2 # whoami
3 root

```

۲ بخش دوم

۱.۲ مقدمه

قبل از حل سوال، باید به کمک دستوراتی مانند `file` یا `checksec` مکانیزم‌های امنیتی اعمال شده روی فایل باینری را تحلیل کرده و درباره‌ی آن اطلاعات کافی به دست آورده و بدانید که چطور باید علیه آن‌ها عمل کنید. در نهایت برای اینکه بتوانید کد را تحلیل کنید، می‌توانید از ابزارهایی مانند `gdb` یا `radare2` استفاده کرده و جریان برنامه و بخش‌های آن را بررسی کنید و رفتار برنامه را نیز به دقت زیر نظر بگیرید. همچنین می‌توانید با `ghidra` و یا سایت `dogbolt` (در صورت دسترسی) برای `disassemble` کردن فایل باینری استفاده کنید تا دیدگاه کافی نسبت به کد اصلی داشته باشید. علاوه بر این می‌توانید از ابزارهایی مانند `hexdump`، `objdump` و یا `strings` برای تحلیل بخش‌های حافظه و یا حتی به دست آوردن `RoP chain` استفاده کنید. همچنین می‌توانید از کتابخانه‌ی `pwntools` در پایتون برای به دست آوردن `RoP chain`، دسترسی به فایل یا سرور و حتی `payload` نهایی خود استفاده کنید. برای آشنایی بیشتر با ابزارهای مورد استفاده، مستنداتی در پوشه‌ی `docs` در دسترس شما قرار داده شده است:

- `gdb`: فایل `gdb-manual.pdf` — راهنمای کامل رسمی GNU GDB
- `radare2`: فایل `radare2-book.pdf` — کتاب رسمی `radare2`
- `ghidra`: فایل `ghidra-intro-slides.pdf` — اسلایدهای معرفی و آموزش Ghidra

نکته: به شدت توصیه می‌شود برای حل چالش‌های اول و دوم، از `pwntools` استفاده کنید.

۲.۲ راه اندازی محیط

در بخش‌های دوم و سوم این تمرین به منظور فراهم کردن یک محیط یکسان برای `exploit` کردن آسیب‌پذیری‌ها، داکر فایلی در اختیار شما قرار خواهد گرفت تا بتوانید محیط مسئله را روی رایانه‌ی شخصی خود داشته و تست کنید. این داکر فیل فقط برای تمرین شماسست و تنها در صورتی که روی سرورهای مقصد به پرچم دست یابید نمره‌ی بخش‌های مربوطه را کسب می‌کنید. همچنین برای اینکه داکر فیل به خوبی روی سیستم شما اجرا شود، مطمئن شوید که معماری سیستم شما `x86` باشد. به جهت راه اندازی محیط بر روی رایانه شخصی کافیست پس از نصب ابزار `docker`، به پوشه‌ی تمرین رفته و آن را `build` و نهایتاً `run` کنید. با این دستور محیط تمرین روی سیستم شما بالا آمده و با دستور `nc localhost [port]` می‌توانید به آن متصل شوید. همچنین در صورت نیاز می‌توانید با کمک دستور `docker exec` از محیط داکر برای بررسی سوالات و بهره‌برداری از آسیب‌پذیری‌ها استفاده کنید.

۳.۲ هدف

در این چالش، شما به یک سیاره می‌روید و با صاحب سیاره آشنا می‌شوید. شما روی سیاره باید یک سری کارها انجام دهید تا صاحب سیاره را شکست دهید. برای اینکه این کار را بکنید، باید صاحب سیاره را وادار کنید تا شکست را قبول کند و باید با رسیدن به `flag.txt` و خواندن محتوای آن، به این هدف برسید. برای حل، حتماً مقدمه را بخوانید و قبل از هر کاری با مکانیزم‌های امنیتی فایل آشنا شوید، چون کار را برای شما سخت می‌کند. برای رسیدن به پرچم، نیاز دارید تا به مطالب کلاس مسلط باشید و انواع اورفلو و دسترسی به شل را بدانید. برای اتصال به این چالش از دستور زیر استفاده کنید:

```
nc 185.205.203.123 2003
```

اکسپلویت را با بررسی، دیکامپایل و دیباگ فایل باینری شروع کنید. شما باید با بهره‌برداری از آسیب‌پذیری برنامه‌ی داده شده، به شل دسترسی پیدا کنید و پرچم موجود در ماشین را چاپ کنید. احتمالاً در رسیدن به پرچم پس از سرریز بافر، نیاز به استفاده از کدها و گجت‌های خود فایل باینری هم خواهید داشت.

۳ بخش سوم

۱.۳ هدف

در این چالش، شما با یک برنامه‌ی تعاملی سروکار دارید که کارش گرفتن، ویرایش، مشاهده و آزادسازی بخشی از حافظه است. شما باید به‌طور کامل با این برنامه تعامل داشته باشید و همچنین نیاز دارید بدانید که مدیریت حافظه‌ی پویا (heap) و ساختار حافظه چگونه عمل کرده و چه نقص‌هایی دارد. همچنین باید روش‌های مختلف دسترسی به شل را امتحان کنید تا بالاخره بتوانید به محتوای پرچم برسید.

برای اتصال به این چالش از دستور زیر استفاده کنید:

`nc 185.205.203.123 2004`

راهنمایی: این سوال ساده نیست و می‌تواند بسیار وقت‌گیر و نکته‌دار باشد و برای همین باید وقت زیادی صرف کنید و برای رسیدن به پرچم تلاش کنید.

۴ بخش چهارم - سوالات تئوری

۱.۴ پیش‌زمینه

شما در درس با مدل BLP آشنا شدید که محرمانگی را با قوانین no read up و no write down اعمال می‌کند. در این سوال، مدل دومی معرفی می‌شود که در کنار BLP کار می‌کند.

مدل صحت Biba

در حالی که BLP از نشت اطلاعات محرمانه به سطوح پایین‌تر جلوگیری می‌کند، هیچ چیزی درباره کیفیت یا قابل اعتماد بودن داده نمی‌گوید. این سناریو را در نظر بگیرید:

یک تحلیل‌گر معتمد، یک پست وبلاگی تأیید نشده و تصادفی می‌خواند، سپس آن را در یک گزارش رسمی طبقه‌بندی شده می‌نویسد. BLP با این کار مشکلی ندارد اما واضحاً این رخداد می‌تواند مشکل‌ساز باشد.

مدل Biba به حل این مشکل با اختصاص یک **سطح صحت** به هر عامل و منبع می‌پردازد که نشان می‌دهد چقدر آن موجودیت قابل اعتماد است. مدل Biba دو قانون اعمال می‌کند که دقیقاً عکس BLP هستند:

قانون no read down عامل S می‌تواند منبع O را بخواند تنها اگر سطح صحت S **حداکثر به اندازه سطح O** باشد:

$$I(S) \leq I(O) \implies \text{خواندن مجاز است}$$

شهود: اگر یک عامل با صحت بالا از منبعی با صحت پایین بخواند، کارهای آینده‌اش بر ورودی غیرقابل اعتماد مبتنی می‌شود و ممکن است خروجی‌هایش «آلوده» شوند.

قانون no write up عامل S می‌تواند به منبع O بنویسد تنها اگر سطح صحت S **حداقل به اندازه سطح O** باشد:

$$I(S) \geq I(O) \implies \text{نوشتن مجاز است}$$

شهود: یک عامل کمتر قابل اعتماد نباید بتواند یک منبع با اعتماد بیشتر را تغییر دهد چون این کار ممکن است داده‌های باکیفیت را خراب کند.

نکته کلیدی: Biba و BLP جریان اطلاعات را در **جهت‌های مخالف** اعمال می‌کنند. این یعنی یک عملیات نوشتن که کاملاً تحت BLP قانونی است ممکن است تحت Biba غیرقانونی باشد و بالعکس. در سیستمی که هر دو را اعمال می‌کند، هر عملیاتی باید هر دو سیاست را به‌طور همزمان برآورده کند.

۲.۴ توصیف سامانه

سازمانی در سه لایه کنترل دسترسی را اعمال می کند:

- مدل BLP برای محرمانگی
 - مدل Biba برای صحت
 - مدل RBAC برای اعمال محرمانگی با توجه به نقش هر عامل
- هر موجودیت دو سطح مستقل برای BLP و Biba دارد:

$$\text{برچسب کامل} = \left(\underbrace{L}_{\text{سطح محرمانگی BLP}} \mid \underbrace{I}_{\text{سطح صحت Biba}} \right)$$

سطوح محرمانگی (BLP)، از پایین به بالا (L):

Unclassified $\prec$ Confidential $\prec$ Secret $\prec$ Top Secret

سطوح صحت (Biba)، از پایین به بالا (I):

Low $\prec$ Medium $\prec$ High $\prec$ Critical

عامل ها

Subject	BLP Label	Biba Label	Roles
Alice	Secret	High	Analyst, Writer
Bob	Top Secret	Medium	Analyst
Carol	Confidential	Critical	Auditor, Writer
Dana	Top Secret	High	Admin, Writer
Eve	Unclassified	Critical	Writer

منبع ها

Object	BLP Label	Biba Label	Required Role
File F_1	Secret	High	Analyst
File F_2	Confidential	Medium	Auditor
File F_3	Top Secret	High	Analyst
Report R_1	Secret	High	Writer
Report R_2	Confidential	Critical	Writer
Log L_1	Unclassified	Critical	Auditor
Database D_1	Top Secret	High	Admin

۳.۴ سوالات

الف، بررسی BLP (۱۰ امتیاز)

برای هر عملیات زیر، مجاز یا ممنوع بودن را با ذکر دلیل مشخص کنید.

۱. آلیس فایل F_1 را می خواند.

۲. ایو فایل F_3 را می خواند.

۳. باب در فایل F_1 می نویسد.

ب، بررسی Biba (۱۰ امتیاز)

برای هر عملیات زیر، مجاز یا ممنوع بودن را با ذکر دلیل مشخص کنید.

۱. دانا فایل F_2 را می خواند.

۲. باب گزارش R_2 را می خواند.

۳. کارول در گزارش R_1 می نویسد.

ج، بررسی همزمان Biba و BLP (۲۰ امتیاز)

برای هر عملیات زیر، مجاز یا ممنوع بودن را با اعمال شرطهای هر دو مدل، با ذکر دلیل مشخص کنید.

۱. آلیس فایل F_3 را می خواند.

۲. باب گزارش R_2 را می خواند.

۳. دانا در L_1 می نویسد.

د، لایه RBAC (۱۰ امتیاز)

فراتر از BLP و Biba، هر منبع به یک نقش خاص نیاز دارد. یک عامل باید علاوه بر گذراندن هر دو بررسی BLP و Biba، نقش موردنیاز را داشته باشد. رد شدن در هر لایه ای برای مسدود کردن دسترسی کافی است.

دسترسی مجاز است $\implies RBAC \wedge BLP \wedge Biba$

عملیاتهای زیر را تحت هر سه سیاست ارزیابی کنید. برای هر کدام مشخص کنید کدام لایه ها عملیات را رد می کنند (در صورت وجود):

۱. باب فایل F_3 را می خواند.

۲. کارول فایل F_2 را می خواند.

۳. کارول گزارش R_2 را می نویسد.

ه، سناریوی جریان اطلاعات (۱۵ امتیاز)

توالی رویدادهای زیر را در نظر بگیرید:

- گام ۱: آلیس F_1 را می خواند
 گام ۲: آلیس یک گزارش جدید R_{new} ، با برچسب Secret | High، می نویسد
 گام ۳: کارول R_{new} را می خواند

۱. هر سه گام را تحت قوانین BLP بررسی کنید. آیا این جریان اطلاعات امکان برقراری دارد؟

۲. در صورتی که پاسخ قسمت قبل مثبت است، چه برچسب‌های دیگری می توان برای R_{new} قرار داد؟ در صورتی که پاسخ منفی است، آیا برچسب گذاری مجدد R_{new} مشکل را حل می کند؟

۳. تحلیلی مشابه قسمت‌های قبل برای قوانین Biba در این جریان اطلاعات ارائه دهید. از این تحلیل و سوال بخش قبل چه نتیجه‌ای می گیرید؟

۵ تحویل دادنی‌ها

شما باید برای هر بخش، اسکرپت خود برای بهره‌برداری از آسیب‌پذیری سوال را به همراه یک ویدیو جامع برای هر بخش، که شامل توضیح اسکرپت و نحوه‌ی رسیدن به اطلاعات لازم برای حل و ساخت شل است، ارسال کنید. ویدیوهای خود را در سامانه‌ی اشتراک گذاری فایل درس قرار داده و فقط لینک آن‌ها را همراه با hash ویدیو در cw ارسال کنید. ساختار فایل زیپ ارسالی شما با نام **ce441-hw1-SID** باید به شکل زیر باشد:

```
1 xploit1-1.c
2 xploit1-2.c
3 xploit1-3.c
4 xploit1-4.c
5 xploit1-5.c
6 exploit2.py
7 exploit3.py
8 urls.txt
9 theory.pdf
```

لازم است در گزارش به‌طور خلاصه مراحل‌ی که طی کرده‌اید را به همراه پرچم نهایی (پرچم روی سرور) گام به گام ذکر کنید. همچنین توضیحات مورد نیاز برای نحوه‌ی اجرای اسکرپت‌ها و پیش‌نیازهای آن را نیز به‌طور کامل در گزارش ذکر کنید. بخش چهارم نیازمند ارائه‌ی پاسخ کتبی در قالب **theory.pdf** است. دقت کنید که اسکرپت‌های شما باید به‌صورت مستقل توسط ما اجرا شده و به پرچم برسد تا نمره‌ی آن بخش را کسب کنید.

در صورت داشتن هرگونه سوال در مورد این تمرین می‌توانید با ایمیل **m.hadadian76@sharif.edu** یا تالارهای گفتگوی درس در cw در ارتباط باشید.